

Backend et NoSQL

Guillaume Guex guillaume.guex@unil.ch

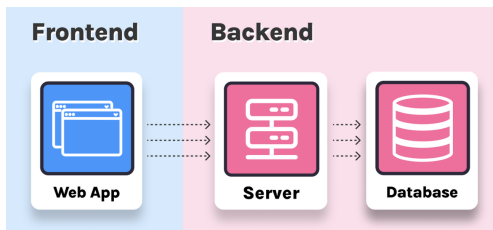
Base de données en backend

Lorsque l'on développe pour le Web (site ou application), on divise les tâches entre le **frontend** et le **backend**.

- Le frontend utilise du HTML, CSS et JavaScript et gère :
 - Les liens.
 - La mise en page.
 - Le graphisme.
 - Les animations.
 - Les interactions.
- Le backend peut être écrit en différents langages (PHP, Python, JavaScript, etc.) et gère :
 - Le management de l'app ou du site.
 - Le stockage.
 - Le débogage.
 - Les API.
 - Les bases de données.

La base de données en backend

La (ou les) **base de données d'un site web** est donc gérée par le serveur en backend. C'est dans là qu'on trouve les différentes requêtes SQL qui vont être nécessaires au bon fonctionnement du site ou de l'app.



Source : <https://v1.scrimba.com/articles/frontend-backend-or-fullstack/>

Exemple de code JavaScript

Avec **Node.js**, on peut installer un connecteur MariaDB avec npm `install mariadb`. La communication est ensuite possible :

```
const mariadb = require('mariadb');
const pool = mariadb.createPool({
  host: 'mydb.com',
  user: 'myUser',
  password: 'myPassword',
  connectionLimit: 5
});
async function asyncFunction() {
  let conn;
  try {
    conn = await pool.getConnection();
    const rows = await conn.query("SELECT 1 as val");
    console.log(rows); // [ {val: 1}, meta: ... ]
    const res = await conn.query("INSERT INTO myTable value (?, ?)", [1, "mariadb"]);
    console.log(res); // { affectedRows: 1, insertId: 1, warningStatus: 0 }

  } catch (err) {
    throw err;
  } finally {
    if (conn) conn.end();
  }
}
asyncFunction().then(() => {
  pool.end();
});
```

Les systèmes de base de données NoSQL

On dénomme les système de gestion de bases de données (SGBD) qui ne sont pas nécessairement relationnels par **SGBD NoSQL** (qui peut vouloir dire *Non-SQL* mais aussi *Not only SQL*).

Ces systèmes existent depuis longtemps, mais sont redevenus populaires depuis les années 2000.

Avec l'essor du Web 2.0, l'explosion des données transférées et l'utilisation de clusters, les besoins des grandes compagnies internet de disposer de SGBD rapides, flexibles et horizontaux devient une nécessité (ce que ne font pas les SGBDR).

Les SGBD SQL :

- **Schéma rigide** fait en amont.
- **Langage standardisé (SQL)**.
- **Verticalité**.
- Transactions axées sur la **cohérence** (ACID).

Les SGBD NoSQL :

- **Schéma flexible** évolutif.
- **Nombreux langages différents**.
- Support d'une **variété de formats de données**.
- **Horizontalité**.
- Transactions axées sur la **disponibilité** (BASE).

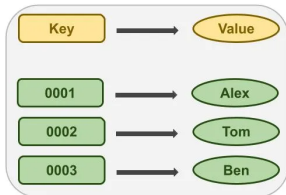
Les SGBD NoSQL se divisent selon **plusieurs modèles d'organisation des données**. Les principaux sont :

- Le modèle clé-valeur.
- Le modèle de documents.
- Le modèle graphe.
- Le modèle à colonnes larges.

De plus, ils existent des systèmes multi-modèles, permettant la gestion de différentes architectures de données.

Le modèle clé-valeur

Le **modèle clé-valeur** est très simple : chaque donnée est référencée par une clé unique.



Code Redis (Bash) :

```
SET user:123 "Alice" # Stocker une valeur
GET user:123         # Récupérer la valeur
DEL user:123          # Supprimer une valeur
```

Avantages : simple, rapide, scalabilité horizontale, données flexibles.

Inconvénients : très (trop ?) simple, transactions limitées, aucune gestion relationnelle.

Exemples d'utilisation : paniers, métadonnées, information de sessions.

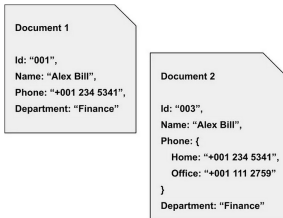
Exemples de SGBD : Redis, DynamoDB.

Source de l'image :

<https://ganganichamika.medium.com/deep-dive-into-nosql-database-types-80340598124>

Le modèle de documents

Le **modèle de documents** stocke une série de documents semi-structurés, similaires à des JSON.



Code MongoDB (JavaScript) :

```
// Insertion d'un document dans la collection "products"
db.products.insertOne({
  name: "Laptop",           // Nom du produit
  price: 1200,              // Prix en dollars
  tags: ["electronics", "computer"], // Catégories associées
  stock: 50                 // Quantité en stock
});

// Recherche des documents où le prix est supérieur à 1000
db.products.find({
  price: { $gt: 1000 }      // $gt signifie "greater than" (supérieur à)
});
```

Avantages : flexible, scalabilité horizontale, support natif pour JSON, bonne compatibilité avec les API.

Inconvénients : transactions limitées, difficile de modéliser des relations complexes.

Exemples d'utilisation : applications web, profiles, catalogues.

Exemples de SGBD : MongoDB, Couchbase.

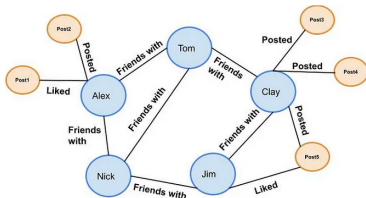
Source de l'image :

<https://ganganichamika.medium.com/deep-dive-into-nosql-database-types-80340598124>

Le modèle graphes

Le **modèle graphes** permet un stockage de données sous forme de noeuds (entités), reliés entre eux par des arêtes (relations).

Code Neo4j (Cypher) :



```
// Exemple 1 : Insérer des nœuds et relations
CREATE (alice:Person {name: "Alice", age: 30}),
      (bob:Person {name: "Bob", age: 25}),
      (eve:Person {name: "Eve", age: 35}),
      (alice)-[:FRIEND]->(bob), // Alice -> Bob
      (bob)-[:FRIEND]->(eve), // Bob -> Eve
      (alice)-[:WORKS_WITH]->(eve); // Alice -> Eve

// Exemple 2 : Trouver les amis d'Alice
MATCH (alice:Person {name: "Alice"})-[:FRIEND]->(friend)
RETURN friend.name, friend.age; // Nom et âge des amis

// Exemple 3 : Trouver les collègues d'Alice
MATCH (alice:Person {name: "Alice"})-[:WORKS_WITH]->(colleague)
RETURN colleague.name, colleague.age; // Nom et âge des collègues
```

Avantages : puissant, flexible, bonne représentation de relations complexes, adapté aux requêtes de type chemin.

Inconvénients : complexe à construire et à étendre.

Exemples d'utilisation : réseaux sociaux, recommandations, systèmes de transactions et financiers.

Exemples de SGBD : Neo4j, Neptune, OrientDB.

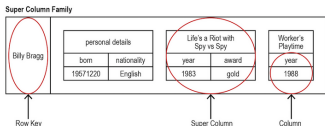
Source de l'image :

<https://ganganichamika.medium.com/deep-dive-into-nosql-database-types-80340598124>

Le modèle colonnes larges

Le **modèle colonnes larges** oriente le stockage par colonne. Il existe des "super colonnes" pouvant en contenir d'autres.

Code Cassandra (SQL) :



```
-- Création de la table "page_views"
CREATE TABLE page_views (
    user_id UUID,           -- ID utilisateur (partition clé)
    page TEXT,              -- Nom de la page
    view_time TIMESTAMP,    -- Heure de la visite (clustering clé)
    PRIMARY KEY (user_id, view_time) -- Tri par view_time dans la partition
);

-- Insertion de données
INSERT INTO page_views (user_id, page, view_time)
VALUES (uuid(), 'HomePage', '2023-12-10 10:00:00');

INSERT INTO page_views (user_id, page, view_time)
VALUES (uuid(), 'ProfilePage', '2023-12-10 10:05:00');

-- Récupération des pages visitées par un utilisateur, triées par heure
SELECT page, view_time
FROM page_views
WHERE user_id = <UUID-user>;
```

Avantages : puissant pour les données massives, horizontalité, SQL.

Inconvénients : peu dynamique, modèle relationnel limité.

Exemples d'utilisation : big data, business intelligence.

Exemples de SGBD : Cassandra, HBase, Bigtable.

Source de l'image : <https://neo4j.com/blog/aggregate-stores-tour/>